Resumen Conceptos y Paradigmas de Lenguajes de Programación

Clase 1

Introducción y evaluación de lenguajes

Introducción

Un lenguaje de programación es un lenguaje formal diseñado para expresar algoritmos y describir procesos computacionales que pueden ser ejecutados por una máquina.

Proporciona un conjunto de reglas sintácticas y semánticas que permiten especificar instrucciones que serán interpretadas o compiladas en operaciones ejecutables por un sistema informático.

Puede seguir o respetar uno o varios paradigmas de programación, que guían y dictan cómo se escriben y conceptualizan los programas.

Un paradigma de programación indica una forma de pensar la programación. Define un modelo, métodos de realizar cómputos y la manera en que se deben estructurar y organizar las tareas que debe llevar a cabo un programa.

Ejemplos:
Se tiene el paradigma imperativo como la programación estructurada, la orientada a objetos; el paradigma declarativo que incluye la programación funcional, la programación lógica.

Principales componentes

Sintaxis

Es el conjunto de reglas sintácticas y gramaticales que determinan cómo se deben estructurar las instrucciones del lenguaje.
Define la forma válida de los programas
Suele describirse mediante gramáticas formales (por ejemplo, BNF o EBNF)

Semántica

Define el **significado** de las construcciones sintácticas

- Semántica operacional: Describe cómo se ejecuta un programa paso a paso.
- Semántica denotacional: Asocia construcciones del lenguaje con objetos matemáticos.

- Semántica axiomática: Permite razonar formalmente sobre la corrección de programas.

Sistema de tipos

Establece reglas sobre los tipos de datos y su uso.
Funciones principales:

1. Detectar errores
2. Garantizar consistencia
3. Mejorar seguridad del programa
   Ejemplos:

- tipado estático vs. dinámico
- tipado fuerte vs. débil

Modelo de ejecución

Define cómo se ejecuta un programa.
Compilado: traducido a código máquina antes de ejecutarse
Interpretado: ejecutado por un intérprete y traducido en tiempo de ejecución
Híbrido: bytecode + máquina virtual

Mecanismos de abstracción

Proporcionan **abstracciones computacionales y de complejidad**
Ejemplos:

- Estructuras de datos
- Estructuras de control
- Módulos y funciones
- Clases y objetos
- Concurrencia

Estudio de conceptos de lenguajes de programación

- Aumentar la capacidad para producir software.
- Mejorar el uso del lenguaje
- Elegir mejor un lenguaje
- Facilitar el aprendizaje de nuevos lenguajes
- Facilitar el diseño e implementación de lenguajes

Criterios de evaluación

Criterios para analizar la calidad del diseño de un lenguaje de programación:

- Simplicidad y legibilidad
- Claridad en los bindings
- Confiabilidad
- Soporte
- Abstracción
- Otogonalidad
- Eficiencia

Ejemplos

En los ejemplos siguientes se observan los criterios de evaluación aplicados, relacionados con los mencionados anteriormente.
Textos obtenidos de trabajos finales de promoción

Pregunta a docentes de Cátedra de Lenguaje C:
Suele observarse que el código de C **no es legible** ¿Cree que esto se debe a la naturaleza del lenguaje o a malos hábitos de programación? ¿Cuál cree que sea la causa?

Rta 1:
*La sintaxis de C permite que pueda escribirse **código muy compacto**. Los alumnos deberían salir preparados para **leer código de otros sin problemas**. Muchos de los proyectos más grandes escritos en C tienen estrictas normas de estilo y esto permite que a pesar de ser compacto, el código **sea suficientemente expresivo** para todos.*

Rta 2:
A la naturaleza del lenguaje sobre todo. El permitir código conciso y…..

Pregunta a docentes de Cátedra lenguaje Ruby:
¿Qué opina de la sintaxis de Ruby?

Rta:
*La sintaxis de Ruby es, a mi parecer, **simple y elegante.** ..... y por otro lado es muy fácil desarrollar con él porque es muy conciso **y preciso** a la hora de implementar cualquier clase de funcionalidad*

Pregunta a docentes de Cátedra lenguaje Ruby:
¿Usted considera que Ruby es un buen ejemplo de un lenguaje que presenta **Ortogonalidad?.** ¿A qué se debe?

Rta:
*A mi parecer sí. El simple hecho de que **toda sentencia del lenguaje sea una expresión** (incluso las estructuras de control) permite realizar combinaciones y/o composiciones sin ninguna clase de limitación. Otro factor que favorece la ortogonalidad es que casi todo en este lenguaje son objetos; no recuerdo fácilmente casos en los cuales haya tenido problemas por encontrarme con algo que no sea/se comporte como un objeto.*

Pregunta a docentes de Cátedra lenguaje Python:
¿Piensa que el lenguaje Python es indicado para iniciar a los alumnos en la programación?

Rta:
*Si, Python es **un lenguaje simple y fácil de enseñar**. Es un lenguaje de tipado dinámico pero es fuertemente tipado, Es un lenguaje **ortogonal** porque se pueden combinar sus componentes. Su código es **legible**, **confiable** por ser fuertemente tipado y proveer manejo de excepcioenes,*

Simplicidad y Legibilidad

Los lenguajes simples y legibles:

- producen programas fáciles de escribir y de leer.
- fáciles a la hora de aprenderlo o enseñarlo
- con sintaxis clara y pocas reglas complejas
- evitan ambigüedades
  Permiten:
- En escritura: expresividad, rapidez al programar.

- En legibilidad: favorece el mantenimiento del software, trabajo en equipo y revisión de código.
  Ejemplo de cuestiones que atentan contra esto:
- Muchas componentes elementales, operadores sobrecargados
- El mismo concepto semántico -- distinta sintaxis
- Distintos conceptos semánticos -- misma notación sintáctica

Claridad en los bindings

Es la asociación entre un nombre y una entidad del programa.
Ejemplos:

- nombre de variable y su valor
- nombre de función y su implementación
  Claridad en los bindings implica:
- saber cuándo y hasta cuando ocurre la asociación
- reglas de scope (alcance) claras y no ambiguas.
  Momentos posibles:
- Definicion del lenguaje
- Implementación del lenguaje
- En escritura del programa
- Compilación
- Cargado del programa
- En ejecución

Tipos de binding:

- Estático(en compilación)
- Dinámico (ejecución)

Confiabilidad

Está relacionada con la seguridad.

Un lenguaje es confiable cuando:

- Evita errores comunes
- Ofrece tipado fuerte
- Detecta errores en compilación
- Previene comportamientos inesperados
  Implica:
- Chequeo de tipos: Cuanto antes se encuentren errores, menos costo resulta realizar los arreglos que se requieran.

- Manejo de excepciones: La habilidad para interceptar errores en tiempo de ejecución, tomar medidas correctivas y continuar.

Soporte

Un lenguaje con buen soporte facilita:

- Aprendizaje
- Desarrollo
- Mantenimiento
  El soporte puede consistir en:
- Compiladores e interpretes
- Bibliotecas
- Frameworks
- Herramientas de depuración
- Comunidad de desarrolladores
- Documentación
- Tutoriales

Abstracción

Permite en programación:

- Representar conceptos complejos mediante estructuras más simples.
- Definir y usar estructuras u operaciones complicadas de manera que sea posible ignorar muchos detalles.
- Ocultar detalles de implementación.
- Reutilizar código.
- Manejar complejidad
  Ejemplos:
- Abstracción de datos: tipo de datos abstractos, clases
- Abstracción en procesos: funciones, módulos, interfaces.

Ortogonalidad

Significa que las características del lenguaje funcionen de manera independiente y consistente entre sí.
Significa en programación:

- Que un conjunto pequeño de constructores primitivos puedan combinarse sin restricciones mediante un conjunto consistente y uniforme de reglas de combinación.
- Cada combinación es legal y con sentido.
  Ejemplos de problemas de ortogonalidad:

- "Funciona sólo con enteros"
- "En este caso no se puede usar dentro de una función"
- "De este tipo no se puede retornar"
  La ortogonalidad reduce:
- Excepciones en las reglas
- Complejidad del lenguaje

Eficiencia

La eficiencia hace referencia al uso eficiente de los recursos computacionales por parte del programa generado por el lenguaje de programación.
Afecta cuestiones cómo:

- Tiempo y espacio
- Esfuerzo humano
- Optimización
  Puede medirse:
- tiempo de ejecución
- Uso de memoria
- Consumo de CPU

Sintaxis

Sintaxis y semántica

Un lenguaje de programación ofrece una notación formal para describir algoritmos a ser ejecutados en una computadora.
Requiere de una sintaxis - si es válido el programa - y semántica - qué significa - tanto para la programación como para su traducción.
Sintaxis: conjunto de reglas que definen cómo componer letras, dígitos y otros caracteres para formar los programas.
Semántica: conjunto de reglas para dar significado a los programas sintácticamente válidos.

Permite escribir programas correctos y válidos sintácticamente. Establece reglas para combinar componentes básicas, llamadas **word** y formar sentencias y programas.
Elementos que la componen:

- alfabeto o conjunto de caracteres
- identificadores
- operadores
- palabra clave y palabra reservada
- comentarios y uso de blancos
  Contempla cuestiones:

- legibilidad
- verificabilidad
- traducción
- Falta de ambigüedad

Sintaxis

Alfabeto:

- Con qué conjunto de caracteres se trabaja
- La secuencia de bits que compone cada caracter determina la implementación.
  Identificadores
- Por ejemplo conformado por letras y dígitos, que deben comenzar con una letra.
- Si se restringe la longitud se pierde legibilidad.
  Operadores
- Uniformes: con los operadores de suma, resta, etc. La mayoría de los lenguajes utilizan +,-.
- Poco uniformes: otros operadores no hay tanta uniformidad (**|^).
  Comentarios
- Para hacer los programas más legibles.

Palabra clave y palabra reservada

- Array
- Do
- Else
- If
- **Palabra clave o keywords**: son palabras claves que tienen un significado dentro de un contexto.
- **Palabra reservada**: Son palabras que además no pueden ser usadas por el programador como identificador de otra entidad.

Ventajas de su uso: Hacen los programas más legibles y permiten una rápida traducción.

Ejemplos de lenguajes con uso de palabras reservadas:

- C ej.:auto, break, case, char, const, continue, default, do ,double, else, enum, extern, float, for, goto, if, int, etc

Estructura sintáctica

Vocabulario o words

- Conjunto de caracteres y palabras necesarias para construir expresiones, sentencias y programas. Ej: identificadores, operadores, palabras claves, etc.
- Las words no son elementales, se construyen a partir del alfabeto.

Expresiones

- Son funciones que a partir de un conjunto de datos devuelven un resultado.
- Son bloques sintácticos básicos a partir de los cuales se construyen las sentencias y programas.

Sentencias

- Componente sintáctico más importante.
- Tiene un fuerte impacto en la facilidad de escritura y legibildad.

Reglas léxicas y sintácticas

Reglas léxicas

- Conjunto de reglas para formar las "word", a partir de los caracteres del alfabeto.
- Por ej. diferencias entre mayúsculas y minúsculas, símbolo distinto != o <>.

Reglas sintácticas

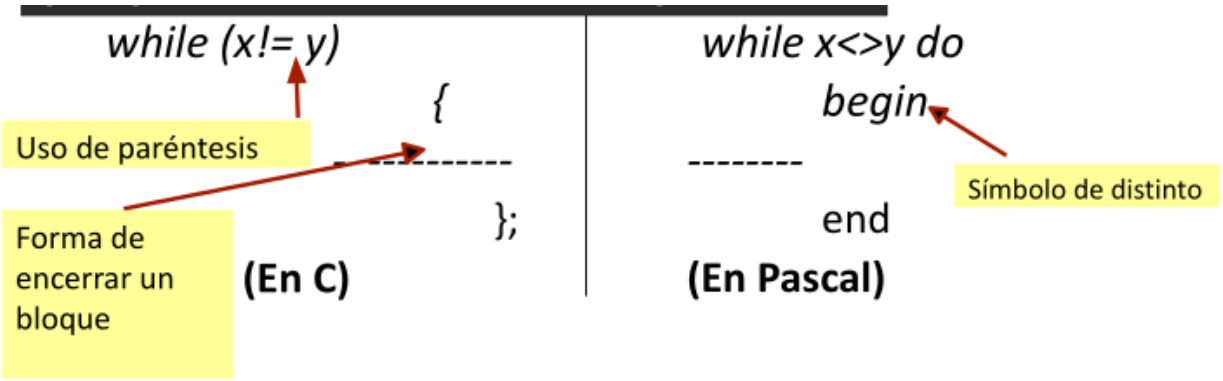
- Conjunto de reglas que definen cómo son las **sentencias** y **expresiones**
- Por ej. En C el if no lleva then, pero en Pascal sí.

Tipo de sintaxis

- Abstracta
  - Se refiere básicamente a la estructura
- Concreta
  - Se refiere a la parte léxica
- Pragmática
  - Uso práctico

Ejemplos

Sintaxis concreta y abstracta



- Son diferentes respecto a la sintaxis concreta, porque existen diferencias léxicas entre ellas.
- Son iguales respecto a la sintaxis abstracta, ya que ambas tienen la misma estructura.

Sintaxis pragmática:

- Ej1.
  - <> es más legible que !=
- Ej2.
  - En C y Pascal {} o begin-end pueden omitirse si el bloque está compuesto de una sola sentencia.
  - while (x!=y) x=y+1
  - Pragmáticamente puede conducir a error ya que si se necesitara agregar una sentencia debe agregarse el begin end o las {} (es decir que no define una única forma de escribirse)

Definición de la sintaxis

Se necesita una descripción finita para definir un conjunto infinito (conjunto de todos los programas bien escritos).

Formas de definir la sintaxis:

- Lenguaje natural: Ej. Fortran
- Gramaticas libres de contextos:
  - Backus y Naun Form: BNF Ej: Algol
  - EBNF
  - Diagramas sintácticos equivalentes a BNF pero más intuitivos

BNF (Backus Naun Form)

- Es una notación formal para describir la sintaxis
- Es un metalenguaje
- Utiliza metasímbolos
  - <, >, ::=, |

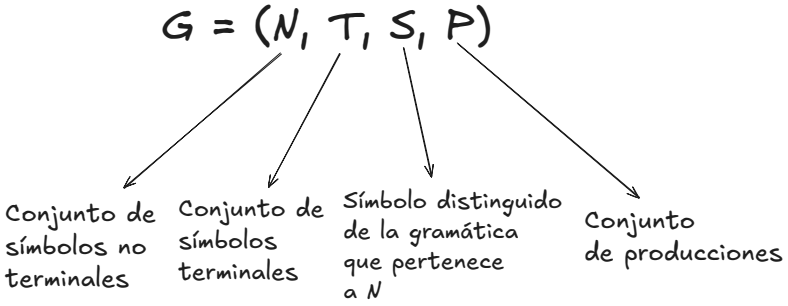
- Define las reglas por medio de "producciones"

Ejemplo:

No terminal	Metasímbolo	Se define como Terminales
<digito>	::=	0|1|2|3|4|516|7|8|9

Gramática

Es un conjunto de reglas finita que define un conjunto infinito de posibles sentencias válidas en el lenguaje.
Una gramática esta formada por una 4-tupla



Parsing y Árboles sintácticos

- Es una oración sintácticamente incorrecta
- No todas las oraciones que se pueden armar con los terminales son válidas
- Se necesita de un Método de análisis (reconocimiento) que permita determinar si un string dado es válido o no en el lenguaje: **Parsing**.

El parse, para cada sentencia construye un "**árbol sintáctico o árbol de derivación**"

Árboles sintácticos

Dos maneras de construirlo:

- Método bottom-up
  - De izquierda a derecha
  - De derecha a izquierda
- Método top-dow
  - De izquierda a derecha
  - De derecha a izquierda

Árbol de derivación

Ejemplo top-down de izquierda a derecha:

N={<oración> => <sujeto><predicado>
<predicado> => <verbo> <art> <objeto>
<sujeto> => Juan | María
<art> => un | una
<objeto> => mascota | perro | gato}
=> Juan <predicado>
=> Juan <verbo><objeto>
=> Juan adopta <objeto>
=> Juan adopta <art><sustan>
=> Juan adopta un <sustan>
=>Juan adopta un perro

- Los compiladores utilizan el parse canónico que es el bottom-up de izquierda a derecha.

Otro ejemplo

- Expresiones simples de uno y dos términos
- Posibles operaciones: + / * y -
- Solo los operandos A, B y C
- Ejemplo de expresiones válidas:
  - A
  - A+B
  - A-C
  - etc.

Producciones recursivas

- Son las que hacen que el conjunto de sentencias escrito sea infinito.
- Ejemplo:
  - <natural> ::= <digito> | <digito><digito> | ..... | <digito>.....<digito>
- Si lo planteamos recursivamente:
  - GN = (N, T, S, P)
    N = { <natural>, <digito> } T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
    S = <natural>
    P = {<natural> ::= <digito> | <digito> <natural>,
    <digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9}

Important

Cualquier gramática que tiene una producción recursiva describe un lenguaje infinito.

- Regla recursiva por la izquierda
  - La asociatividad es por la izquierda
  - El símbolo no terminal de la parte izquierda de una regla de producción, aparece al comienzo de la parte derecha
- Regla recursiva por la derecha
  - La asociatividad es por la derecha
  - El símbolo no terminal de la parte izquierda es una regla de producción, aparece al final de la parte derecha

Gramáticas ambiguas

Una gramática es ambigua si una sentencia puede derivarse de más de una forma.

G= (N, T, S, P)
N = { <id>, <exp>, <asig>} T = {
A,B,C, +,*,-,/,:=}
S = <asig>
P1 = {
<asig> ::= <id> := <exp>
<exp> ::= <exp>+<exp>|<exp>*<exp>|<exp>- <exp>|
<exp>/ <exp>|<id>
<id> ::= A | B | C
}

recursión

Subgramáticas

Sea la gramática para identificadores GI = (N, T, S, P)

N = { <id>, <letra>, <digito>, <otro>}
T = { A, ..., Z, 0, ..., 1}
S = <id>
P = { <id> ::= <letra> | <letra><otro>,
<otro>::= <letra> | <digito> | <letra><otro>
| <digito><otro>,
<letra> :: A | B | C |.....| Z
<digito> ::= 0 | 1 | 2 | | 9 }

- Para definir la gramatica GE de expresiones, se puede utilizar la gramática de números y de identificadores.
- GE se definiría utilizando las sub gramáticas GN y GI

> **Important**
>
> La filosofía de composición es la forma en que trabajan los compiladores

Gramáticas libres de contexto y sensibles al contexto

int y:
y= 22.75

- La gramática libre de contexto no puede verificar:
  - Si x fue declarada antes
  - Si los tipos son compatibles
- No considera lo que se define antes ni después.
- Según nuestra gramática son sentencias sintácticamente válidas, aunque puede suceder que a veces no lo sea semánticamente.

Gramática libre de contexto (CFG)

- Es aquella en la que no realiza un análisis de contexto.
- Describen las sintaxis de los lenguajes como C, Java, Python

Gramática sensible al contexto (CSG)

- Se realizan verificaciones que consideran el contexto del programa (Algol 68)
- Se manejan en la fase del análisis semántico del compilador, para detectar ciertas restricciones semánticas.

- Se pueden utilizar en IA en aplicaciones de machine learning, reconocimiento de voz, en lingüística para especificar reglas para lenguaje natural.

EBNF BNF extendido

- EBNF es una **gramática libre de contexto**
- Simplifica la escritura
- Elimina la recursión y la reemplaza por reiteración

Meta símbolos incorporados:

- [] elemento optativo puede o no estar
- (|) selección de una alternativa
- {} repetición

- 0 o más veces + 1 o más veces

Ejemplo con EBNF

Definición números enteros en BNF y en EBNF:

BNF

<enterosig> ::= + <entero> | - <entero> | <entero>

<entero> ::= <digito>| <entero><digito>

Recursión

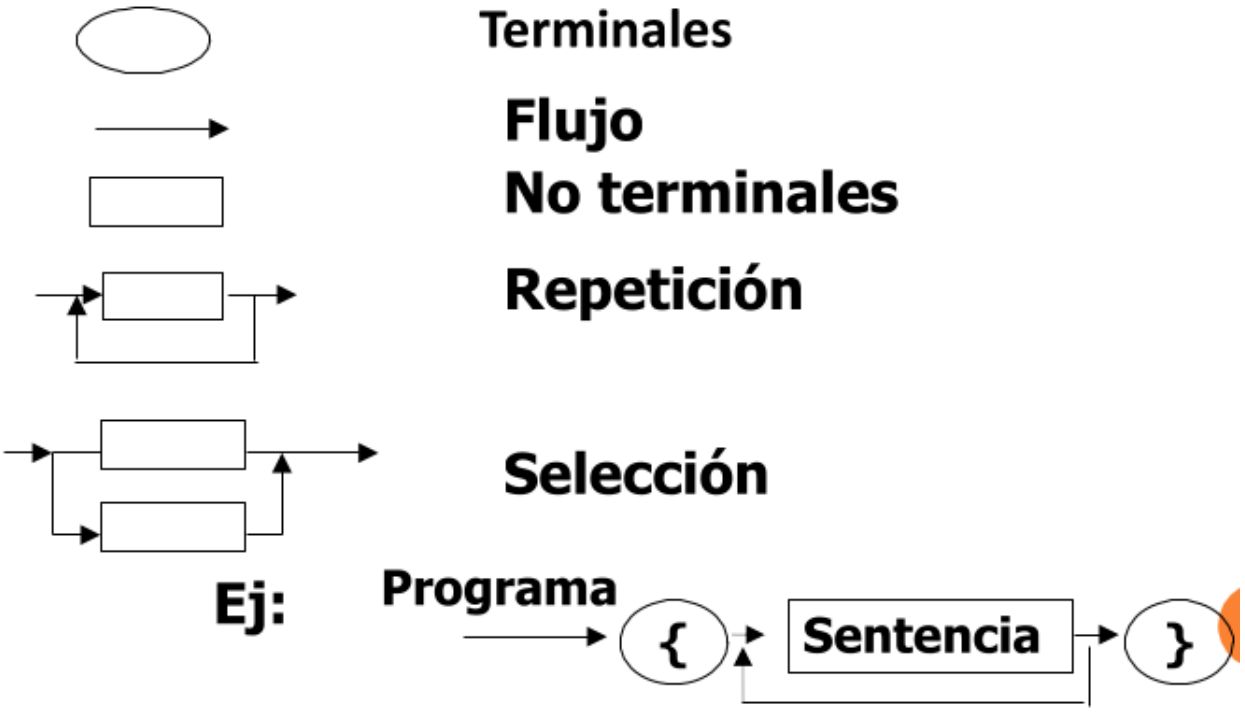
EBNF

<enterosig>::= **[**(+|-)**]** <digito>{<digito>}*

Eliminó la recursión y es más fácil de entender.

Diagramas sintácticos (Conway):

- Es un grafo sintáctico o carta sintáctica
- Cada diagrama tiene una entrada y una salida, y el camino determina el análisis.
- Cada diagrama representa una regla o producción
- Para que una sentencia sea válida, debe haber un camino desde la entrada hasta la salida que la describa.
- Se visualiza y entiende mejor que BNF o EBNF


Terminales
Flujo
No terminales
Repetición
Selección
Ej:
Programa
{
Sentencia
}